

Semantic Analysis

Chapter 5

Outline

- Symbol Tables
- Symbols in the Tiger Compiler
- **Type Checking**

Key Problems in Type Checking

- What are valid type expressions?
 - e.g., int, string, nil, array of int, record, ...
- How to define two types are equivalent?
- What are the type-checking rules?

Types in Tiger Programming Language

- The primitive type: **int**, **string**
- The constructed type: **records** and **arrays** from other types (primitive, record, or array)
- What is the binding of a type identifier?

$typec \rightarrow \text{type } type\text{-}id = ty$

$ty \rightarrow type\text{-}id$

$\rightarrow \{ \} tyfields \{ \}$

$\rightarrow \text{array of } type\text{-}id$

$tyfields \rightarrow \varepsilon$

$\rightarrow id: type\text{-}id \{, id: type\text{-}id \}$

Grammar

```
let type a = {x: int; y: int}  
    type b = a  
    var i : a := ...  
    var j : b := ...  
in i := j  
end
```

Example

Please read and learn the [Appendix Tiger Language Reference Manual](#)

Types in Tiger Programming Language

- What are the bindings of type identifiers?

typec → *type* *type-id* = *ty*

ty → *type-id*

→ '{' *tyfields* '}'

→ *array of type-id*

tyfields → ϵ

→ *id: type-id* {, *id:type-id*}

```
typedef struct Ty_ty_ *Ty_ty;
struct Ty_ty_ {
    enum {Ty_record, Ty_nil, Ty_int, Ty_string,
          Ty_array, Ty_name, Ty_void} kind;
    union {
        Ty_fieldList record;
        Ty_ty array;
        struct {S_symbol sym; Ty_ty ty;} name;
    } u;
};
```

- **nil:** *Ty_Nil*
- **void:** *Ty_Void*
- When processing mutually recursive types: *Ty_Name(sym, NULL)*
 - a place-holder for the type-name *sym*

```
type list = {first: int, rest: list}
```

Type Equivalence

- **Name equivalence (NE)**: T1 and T2 are equivalent iff T1 and T2 are **identical type names** defined by **the exact same type declaration**.
- **Structural equivalence (SE)**: T1 and T2 are equivalent iff T1 and T2 are composed of **the same constructors** applied in **the same order**.
- Tiger uses name equivalence.

Type Equivalence in Tiger

- Every Tiger-language “record type expression” creates a new (and different) record type.

```
let type a = {x: int; y: int}  
    type b = {x: int; y: int}  
    var i : a := ...  
    var j : b := ...  
in i := j  
end
```

It is illegal
in Tiger.

```
let type a = {x: int; y: int}  
    type b = a  
    var i : a := ...  
    var j : b := ...  
in i := j  
end
```

It is legal
in Tiger.

Namespaces in Tiger

- Tiger has two separate name spaces:
 - **Types**
 - **Functions** and **variables**

```
let type a = int
    var a := 1
in ...
end
```

Both a's
can be used

```
let function a (b: int) = ...
    var a := 1
in ...
end
```

var a hides
function a

Environments for Type Checking

Tiger maintains two environments:

- **Type environment:**

- Maps type symbols to their type objects
 - `symbol` -> `Ty_ty`

- **Value environment:**

- Maps variable symbols to their type objects
 - `symbol` -> `Ty_ty` Why not map to the symbol of the type name?
- Maps function symbols to the type objects of their parameter and result types.
 - `symbol` -> struct {`Ty_tyList` formals, `Ty_ty` result;}

- How to know which `a` is a type and which `a` is a variable?
 - based on syntactic contexts

```
let type a = int
    var a: a := 5
    var b: a := a
in b+a
end
```

Environments

```
typedef struct E_entry_ *E_entry;
struct E_entry_ {
    enum {E_varEntry, E_funEntry} kind;
    union {
        struct {Ty_ty ty;} var;
        struct {Ty_tyList formals; Ty_ty result;} fun;
    } u;
};

E_entry E_VarEntry(Ty_ty ty);
E_entry E_FunEntry(Ty_tyList formals, Ty_ty result);

S_table E_base_tenv(void); /* Ty_ty environment */
S_table E_base_venv(void); /* E_entry environment */
```

Type-Checking for Tiger

- The `Semant` module (`semant.h`, `semant.c`) performs semantic analysis - including type-checking – of abstract syntax.
- The semantic analysis module contains four functions that recur over syntax trees:

```
Struct expty transVar (S_table venv, S_table tenv, A_var v);  
Struct expty transExp (S_table venv, S_table tenv, A_exp a);  
Void transDec (S_table venv, S_table tenv, A_dec d);  
Ty_ty transTY (S_table tenv, A_ty a);
```

- The type-checker is a *recursive function* of the abstract syntax tree. -- **transExp**
- The four functions perform both type-checking and IR Generation.
 - For now, we only consider type-checking

Type-Checking for Tiger

- **Type-Checking expressions**
- Type-Checking declarations
 - Variable declarations
 - Type declarations
 - Function declarations
 - Recursive declarations

Type-Checking Expressions

```
Struct expty transVar (S_table venv, S_table tenv, A_var v);  
Struct expty transExp (S_table venv, S_table tenv, A_exp a);  
Void transDec (S_table venv, S_table tenv, A_dec d);  
Ty_ty transTY (S_table tenv, A_ty a);
```

- transExp: Type-checking expressions: query && update the environments

argument: a value environment `venv`
a type environment `tenv`
an expression `a`

result: a translated expression and its Tiger-language type.

```
struct expty {Tr_exp exp; Ty_ty ty;};
```

- e.g., `a + b`

Type-Checking Expressions: Example

- Tiger's **nonoverloaded** type-checking for '+' expression
 - For **e1 + e2**, the types of both **e1** and **e2** must be **int**
 - the type of the expression is also **int**

```
struct expty transExp(S_table venv, S_table tenv, A_exp a)
{switch(a->kind) {
    ...
    case A_opExp: {
        A_oper oper = a->u.op.oper;
        struct expty left =transExp(venv,tenv,a->u.op.left);
        struct expty right=transExp(venv,tenv,a->u.op.right);
        if (oper==A_plusOp) {
            if (left.ty->kind!=Ty_int)
                EM_error(a->u.op.left->pos, "integer required");
            if (right.ty->kind!=Ty_int)
                EM_error(a->u.op.right->pos,"integer required");
            return expTy(NULL,Ty_Int());
        }...
    }
}
assert(0); /* should have returned from some clause of the switch
*/}
```

Type-Checking Variables, Subscripts and Fields

lvalue $\rightarrow$ *id*
 $\rightarrow$ *lvalue* . *id*
 $\rightarrow$ *lvalue* [*exp*]

- An **l-value** is a location whose value may be read or assigned.
- Variables, procedure parameters, fields of records, and elements of arrays are all l-values.

```
struct expty transVar(S_table venv, S_table tenv, A_var v )
{
    switch(v->kind) {
        case A_simpleVar: {
            E_enventry x = S_look(venv, v->u.simple);
            if (x && x->kind == E_varEntry)
                return expTy(NULL, actual_ty(x->u.var.ty));
            else {
                EM_error(v->pos, "undefined variable %s", S_name(v->u.simple));
                return expTy(NULL, Ty_Int());
            }
        case A_fieldVar:
            ...
    }
}
```

/ A_var is the structure for l-value */*

5.3&5.4 Type-Checking

- Type-Checking expressions
- **Type-Checking declarations**
 - Variable declarations
 - Type declarations
 - Function declarations
 - Recursive declarations

5.4 Type-Checking Declarations

- Environments are constructed and augmented by declarations.
- In Tiger, declarations appear only in a **let** expression.

let *decs* **in** *body* **end**

```
struct expty transExp (S_table venv, S_table tenv, A_exp a) {
    switch(a->kind) {
        ...
        case A_letExp: {
            struct expty exp;
            A_decList d;
            S_beginScope(venv); S_beginScope(tenv);
            for (d = a->u.let.decs; d; d=d->tail)
                transDec(venv,tenv,d->head);
            exp = transExp(venv,tenv,a->u.let.body);
            S_endScope(tenv); S_endScope(venv);
            return exp;
        }
    }...
}
```

- **How is transDec implemented?**

- variable, type, function

5.3&5.4 Type-Checking

- Type-Checking expressions
- **Type-Checking declarations**
 - Variable declarations
 - Type declarations
 - Function declarations
 - Recursive declarations

Variable Declarations

- Recursive declarations v.s. non-recursive declarations
- For example, processing a variable declaration **without a type constraint**, such as

var x := exp

```
void transDec(S_table venv, S_table tenv, A_dec d) {  
    switch(d->kind) {  
        case A_varDec: {  
            struct expty e = transExp(venv, tenv, d->u.var.init);  
            S_enter(venv, d->u.var.var, E_VarEntry(e.ty));  
        }  
        ...  
    }  
    ...  
}
```

Variable Declarations

- Processing a variable declaration **with a type constraint:**

var x : type-id := exp

- It will be necessary to check that the constraint and the initializing expression are compatible.
- Also, initializing expressions of type **Ty_Nil** must be constrained by a **Ty_Record** type (according to the reference manual of Tiger)

Type Declarations

- Nonrecursive type declarations

type type-id = ty

- transTy: translates *A_ty* to *Ty_ty* Recursively

```
void transDec(S_table venv, S_table tenv, A_dec d) {  
    ...  
    case A_typeDec: {  
        S_enter(tenv, d->u.type->head->name, transTy(d->  
u.type->head->ty));  
    }  
    ...  
}
```

Notice: This program fragment *only handles* the type-declaration list of length 1

Function Declarations

function id (tyfields) : type-id = exp

```
void transDec(S_table venv, S_table tenv, A_dec d) {
    switch(d->kind) {
        ...
        case A_functionDec: {
            A_fundec f = d->u.function->head;
            Ty_ty resultTy = S_look(tenv, f->result);
            Ty_tyList formalTys = makeFormalTyList(tenv, f->params);
            S_enter(venv, f->name, E_FunEntry(formalTys, resultTy));
            S_beginScope(venv);
            {A_fieldList l; Ty_tyList t;
              for(l=f->params, t=formalTys; l; l=l->tail, t=t->tail)
                S_enter(venv, l->head->name, E_VarEntry(t->head));
            }
            transExp(venv, tenv, d->u.function->body);
            S_endScope(venv);
            break;
        }
        ...
    }
}
```

Function Declarations

- The above code is a very stripped-down implementation:
 - it handles only the case of a single function;
 - it handles only a function with a result
 - it doesn't handle program errors
 - it doesn't check that the type of the body expression matches the declared result type
 - ...
- **function** $f(a: ta, b: tb) : rt = body$
- ***makeFormalTyList***: traverses the list of formal parameters and returns a list of their types

Recursive Declarations

type list = {first: int, rest: list}

- How to convert this declaration into internal type representations?
- Before adding *list* to the type environment, we need to process *{first: int, rest: list}*
- **Problem:** Processing *{first: int, rest: list}* requires to lookup *list* from the type environment
 - *undefined type!*
- How to handle this problem?

Recursive Declarations

```
struct {  
    S_symbol sym;  
    Ty_ty ty;} name;
```

type list = {first: int, rest: list}

- **Solution:** use two passes:
 - 1. Put all the “headers” in the environment first (though they do not have bodies)
 - What is a header in this example? *type list =*
 - How to enter this header into an environment?
S_enter(tenv, name, Ty_Name(name, NULL));
 - 2. call *transTy* on the “body” of the type declaration
 - that is, the record type expression *{first: int, rest: list}*
 - The *type* that *transTy* returns can then be assigned into the *ty* field within the *Ty_Name* struct.

Recursive Declarations

```
struct {  
  S_symbol sym;  
  Ty_ty ty;} name;
```

- Every cycle in a set of mutually recursive type declarations must pass through a record or array declaration.
- Why?
 - Otherwise, the type checker will never terminate.

```
type a = b  
type b = a
```

It is illegal
in Tiger.

```
type a = b  
type b = {i: a}
```

```
type a = b  
type b = d  
type c = a  
type d = a
```

It is illegal
in Tiger.

contains an illegal cycle:
a -> b -> d -> a

- Illegal cycles should be detected by the type-checker

Recursive Declarations

Mutually Recursive Functions

f call g , g call f

- **Problem:** when we process the body of f , we will encounter g , which that cannot be found in the var env yet.
- **Solution:**
 - First pass: gathers information about the *header* of each function (function name, formal parameter list, return type) but leaves the bodies of the functions untouched.
 - Second pass: processes the *bodies* of all functions with the augmented environment.

Homework

- 5.1 (a, b)